Ответы к защите лабораторной работы № 1

Линейная регрессия и факторный анализ. Appliances Energy Prediction

Содержание

I. Вопросы по коду **3**

1. Загрузка данных: код, параметры и их назначение . . . 3
2. Количество наблюдений, количество признаков и типы столбцов . . . 3
3. Целевая переменная и формирование X и y . . . 4
4. Какие столбцы исключены из признаков и почему . . . 4
5. Проверка пропусков и дубликатов . . . 4
6. Описательные статистики и обнаруженные особенности . . . 5
7. Распределения признаков и целевой переменной . . . 5
8. Выбросы: как исследованы и почему не удалены . . . 6
9. Удаление пропусков и оставшееся число наблюдений . . . 7
10. Категориальные признаки и их кодирование . . . 7
11. Какие признаки масштабируются и где . . . 7
12. Различие fit, transform и fit_transform . . . 8
13. На каких данных обучается предобработка и как исключён тест . . . 8
14. Код корреляционной матрицы и вошедшие столбцы . . . 9
15. Метод корреляции и причина выбора . . . 10
16. Пары с наиболее сильной связью . . . 11
17. VIF: код, входные данные и интерпретация . . . 11
18. Что предложить удалить или преобразовать . . . 12
19. Разделение на train/test и размеры выборок . . . 12
20. Назначение random_state и последствия его изменения . . . 12
21. Подходит ли случайное разбиение для этих данных . . . 13
22. Создание и обучение трёх регрессий . . . 13
23. Явные параметры моделей и параметры по умолчанию . . . 14
24. Подбор alpha: сетки и полученные значения . . . 14
25. Кросс-валидация: код, число фолдов и метрика . . . 15
26. Где обучаются scaler и PCA во время CV . . . 15
27. Pipeline: последовательность и назначение шагов . . . 16
28. Коэффициенты обученных моделей и их различия . . . 16
29. Получение прогнозов и выборка итоговой оценки . . . 18
30. Где вычисляются RMSE, R^2 и MAPE и какие массивы используются . . . 18
31. Как получить RMSE через MSE . . . 19
32. Единицы RMSE, R^2 и MAPE . . . 19
33. Нулевые и малые значения цели и интерпретация MAPE . . . 19
34. Качество на train и test: есть ли переобучение . . . 20
35. Как выбрать лучшую модель при несогласии метрик . . . 20
36. Стандартизация перед PCA и выборка её обучения . . . 21
37. Где создаётся и обучается PCA, что подаётся на вход . . . 21
38. Правило выбора количества компонент . . . 22
39. explained_variance_ratio_ и накопленная дисперсия . . . 22
40. Размеры матриц до и после PCA . . . 23

41. Преобразование теста в главные компоненты . . . 23
42. Нагрузки компонент и наиболее связанные исходные признаки . . . 24
43. Проверка устранения мультиколлинеарности после PCA . . . 25
44. Повторное обучение трёх регрессий на компонентах . . . 25
45. Одинаковы ли тестовые наблюдения и правила оценки . . . 26
46. Итоговая таблица и эффект PCA для каждой модели . . . 26

II. Теория **27**
T1. Линейная регрессия . . . 27
T2. Ridge-регрессия и L2-регуляризация . . . 27
T3. Lasso-регрессия и L1-регуляризация . . . 28
T4. Корреляционная матрица, мультиколлинеарность и VIF . . . 28
T5. Метод наименьших квадратов . . . 29
T6. Коэффициент детерминации R^2 . . . 30
T7. Метрики RMSE, MAE и MAPE . . . 30
T8. Train, validation, test и кросс-валидация . . . 31
T9. Недообучение и переобучение . . . 31
T10. Факторный анализ . . . 32
T11. Метод главных компонент — PCA . . . 32
T12. График каменистой осыпи . . . 33
T13. Зависимая и независимые переменные . . . 33
T14. Остатки, гомоскедастичность и гетероскедастичность . . . 33
T15. Коэффициенты, свободный член и множественная регрессия . . . 34

III. Короткая формулировка результата для начала защиты **35**

Материал относится к **сокращённой версии лабораторной**, коммит **87ab971**. В этой версии используются **25 исходных числовых предикторов** и **11 главных компонент**. Номера вопросов 1–46 соответствуют порядку вопросов по коду; разделы T1–T15 — темам по теории.

Фрагменты с пометкой **«Основной код»** взяты из действующего lab1.py. Ссылки на строки относятся к этому файлу, а не к номерам ячеек Jupyter. Фрагменты **«Дополнительно для защиты»** показывают, как вывести подробности, которые сокращённый notebook отдельно не печатает. Эти вычисления выполнены в изолированной временной папке; их тестовые результаты совпали с лабораторной. Исходная лабораторная и Git-репозиторий при подготовке ответов не изменялись.

Дополнительные команды с base_models, pca_models, X_train и другими переменными предполагают, что выполнены соответствующие ячейки notebook. Добавлять их в сдаваемую работу для использования этого пособия не требуется.

Основные числа, которые нужно помнить

Показатель	Значение
Исходный набор	19 735 строк × 29 столбцов
Пропуски / полные дубликаты	0 / 0
Цель	Appliances, энергия в Wh (Вт·ч)
Подготовленные X и y	X: 19 735 × 25; y: 19 735 значений
Train / test	15 788 / 3947 наблюдений, 80/20
Внешняя / внутренняя CV	5 / 3 фолда
VIF исходных предикторов	17 из 25 > 10; максимум T_out ≈ 150.12
PCA	11 из 25 компонент; 95.7086% дисперсии X
VIF компонент на train	≈ 1 для всех 11 компонент

Показатель	Значение
Выбор по внешнему CV RMSE	Lasso на исходных признаках; отличия от OLS/Ridge очень малы
Её тестовые метрики	RMSE 91.1672 Wh; R^2 0.169442; MAPE 62.8847%

I. Вопросы по коду

1. Загрузка данных: код, параметры и их назначение

Основной код: lab1.py, строки 41–44.

```
source = json.loads(Path("data/source.json").read_text(encoding="utf-8"))
data_file = Path("data") / source["file"]
assert hashlib.sha256(data_file.read_bytes()).hexdigest() == source["sha256"]
raw = pd.read_csv(data_file, parse_dates=["date"])
```

Читается локальный файл data/energydata_complete.csv. Его адрес берётся из data/source.json. Перед чтением вычисляется SHA-256 и сверяется с зафиксированной суммой: так проверяется неизменность конкретной копии набора. Хеш не оценивает качество данных — он проверяет целостность файла.

Явно задан параметр **parse_dates=[“date”]**: pandas преобразует дату в datetime64[ns], чтобы это была временная метка, а не обычная строка. Остальные параметры чтения оставлены стандартными: разделитель — запятая, первая строка содержит имена столбцов, десятичный разделитель — точка. Числовые типы определяются автоматически. Специальный список na_values не нужен: нестандартных обозначений пропусков в этой зафиксированной копии нет.

Сам CSV ранее получен из репозитория автора исходного исследования, соответствующего выбранному набору Appliances Energy Prediction на Kaggle. Работа с локальной копией не требует токена Kaggle и повторного скачивания.

2. Количество наблюдений, количество признаков и типы столбцов

Основной код: lab1.py, строки 45–50.

```
display(raw.head(3))
display(pd.Series({"Строк": len(raw), "Столбцов": raw.shape[1],
                   "Пропусков": raw.isna().sum().sum(),
                   "Дубликатов строк": raw.duplicated().sum(),
                   "rv1 и rv2 совпадают": raw.rv1.equals(raw.rv2)}).to_frame("Значение"))
display(raw.Appliances.describe().to_frame("Appliances, Wh"))
```

len(raw) и **raw.shape[0]** дают число строк — 19 735. **raw.shape[1]** даёт число всех столбцов — 29. Это ещё не количество предикторов модели: сюда входят цель, дата и два случайных контроля. После подготовки **X.shape[1] = 25**.

Типы сначала определяет read_csv. После parse_dates фактически получены: один datetime64[ns] — date; два int64 — Appliances и lights; 26 float64 — температуры, влажности, погода и случайные контроли. Строковых категориальных столбцов в загруженной таблице нет.

Дополнительно для защиты — полный вывод типов:

```
print(raw.shape)
print(X.shape)
print(raw.dtypes)
raw.info()
```

В основном коде числовые столбцы также выделяются через raw.select_dtypes(include=np.number). Получаются 28 столбцов, включая цель и оба случайных контроля.

3. Целевая переменная и формирование X и y

Основной код: lab1.py, строки 70–74.

```
clean = raw.dropna().drop_duplicates().copy()
X = clean.drop(columns=["date", "Appliances", "rv1", "rv2"])
y = clean["Appliances"]
assert y.gt(0).all()
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.20, random_state=42)
```

Цель — **Appliances**, то есть энергия, затраченная бытовыми приборами за интервал наблюдения, в Wh. Это количественная величина. Округление значений до дискретных уровней не превращает задачу в классификацию.

X — таблица предикторов, **y** — одномерная последовательность правильных ответов. Appliances удаляется из X: иначе модель получила бы ответ среди собственных входных данных, что было бы утечкой целевой переменной. До разделения X имеет размер 19 735 × 25, а y — длину 19 735.

4. Какие столбцы исключены из признаков и почему

Исключены четыре столбца:

- **Appliances** — это ответ y, его нельзя оставлять в X.
- **date** — временная метка. В данной версии календарные признаки не создаются, а абсолютная дата напрямую в регрессию не передаётся.
- **rv1, rv2** — искусственные случайные контрольные величины по описанию набора. Они не являются измерениями физического состояния дома. Дополнительно установлено, что их значения полностью одинаковы.

Два одинаковых предиктора создавали бы точную линейную зависимость и бесконечный VIF. Оба удалены по назначению признаков, а не потому, что с ними случайно ухудшилась метрика на тесте. Реальные температуры и погодные показатели до PCA не удалялись: важно сравнить исходное пространство предикторов и его PCA-представление. **Основной код: lab1.py, строки 71–72.**

```
X = clean.drop(columns=["date", "Appliances", "rv1", "rv2"])
y = clean["Appliances"]
```

5. Проверка пропусков и дубликатов

Основной код: lab1.py, строки 46–49.

```
display(pd.Series({"Строк": len(raw), "Столбцов": raw.shape[1],
                   "Пропусков": raw.isna().sum().sum(),
                   "Дубликатов строк": raw.duplicated().sum(),
                   "rv1 и rv2 совпадают": raw.rv1.equals(raw.rv2)}).to_frame("Значение"))
```

raw.isna() возвращает булеву таблицу признаков пропуска. Первый sum() считает пропуски по столбцам, второй — общее число пропущенных ячеек. Для подробного вывода можно использовать **raw.isna().sum()**.

raw.duplicated() помечает повторные полные строки, кроме первого появления. Сумма этих флагов — число удаляемых повторов. Результаты: **0 пропусков и 0 полных дубликатов**. Это не означает, что все значения отдельных признаков различны: одинаковые температуры или уровни потребления нормальны.

Дополнительная проверка временных меток показала 19 735 уникальных дат и отсутствие повторных временных меток. Все последовательные интервалы равны 10 минутам. Семантические ошибки измерений одним duplicated() выявить невозможно: он ищет именно совпадающие строки.

6. Описательные статистики и обнаруженные особенности

Основной код: lab1.py, строки 50–50.

```
display(raw.Appliances.describe().to_frame("Appliances, Wh"))
```

describe() выводит число непустых значений, среднее, выборочное стандартное отклонение, минимум, квартили и максимум. Для Appliances получено:

Статистика	Значение, Wh
count	19735.0000
mean	97.6950
std	102.5249
min	10.0000
25%	50.0000
50%	60.0000
75%	100.0000
max	1080.0000

Среднее 97.69 заметно выше медианы 60: распределение асимметрично вправо. Стандартное отклонение 102.52 сопоставимо со средним; расход существенно колеблется. Максимум 1080 Wh намного выше типичных 50–100 Wh. Дополнительно рассчитанная асимметрия равна **3.386**, 95-й процентиль — **330 Wh**.

Для всех числовых столбцов можно дополнительно вывести:

```
display(raw.select_dtypes(include=np.number).describe().T)
```

Температура, влажность, давление и энергия выражены в разных единицах и имеют разные масштабы. Поэтому перед регуляризацией и PCA применяется StandardScaler. describe() сам по себе не стандартизирует и не очищает данные.

7. Распределения признаков и целевой переменной

Основной код: lab1.py, строки 52–60.

```
numeric = raw.select_dtypes(include=np.number)
fig, axes = plt.subplots(6, 5, figsize=(17, 17))
for ax, column in zip(axes.flat, numeric.columns):
    ax.hist(numeric[column], bins=30, color="#3486a6")
    ax.set(title=column, ylabel="Частота")
for ax in axes.flat[len(numeric.columns):]:
    ax.axis("off")
fig.suptitle("Распределения числовых переменных, включая Appliances", y=1.01)
show_plot("01_distributions.png")
```

Выделяются все 28 числовых столбцов исходной таблицы, в том числе Appliances, rv1 и rv2. Для каждого строится гистограмма с 30 интервалами. Сетка 6 × 5 содержит 30 мест; два лишних графика отключаются. Результат сохранён в results/figures/01_distributions.png.

У Appliances виден длинный правый хвост: небольшое потребление часто, большие пики редки. Нули lights — интервалы без расхода на освещение, а не

отсутствующие данные. Гистограммы rv1 и rv2 совпадают, поскольку столбцы дублируют друг друга. У признаков различаются масштабы и формы.

Эти графики описывают весь набор. Нормальное распределение каждого предиктора не является обязательным условием применения линейной регрессии. По форме гистограммы также нельзя доказать причинность или качество будущей модели.

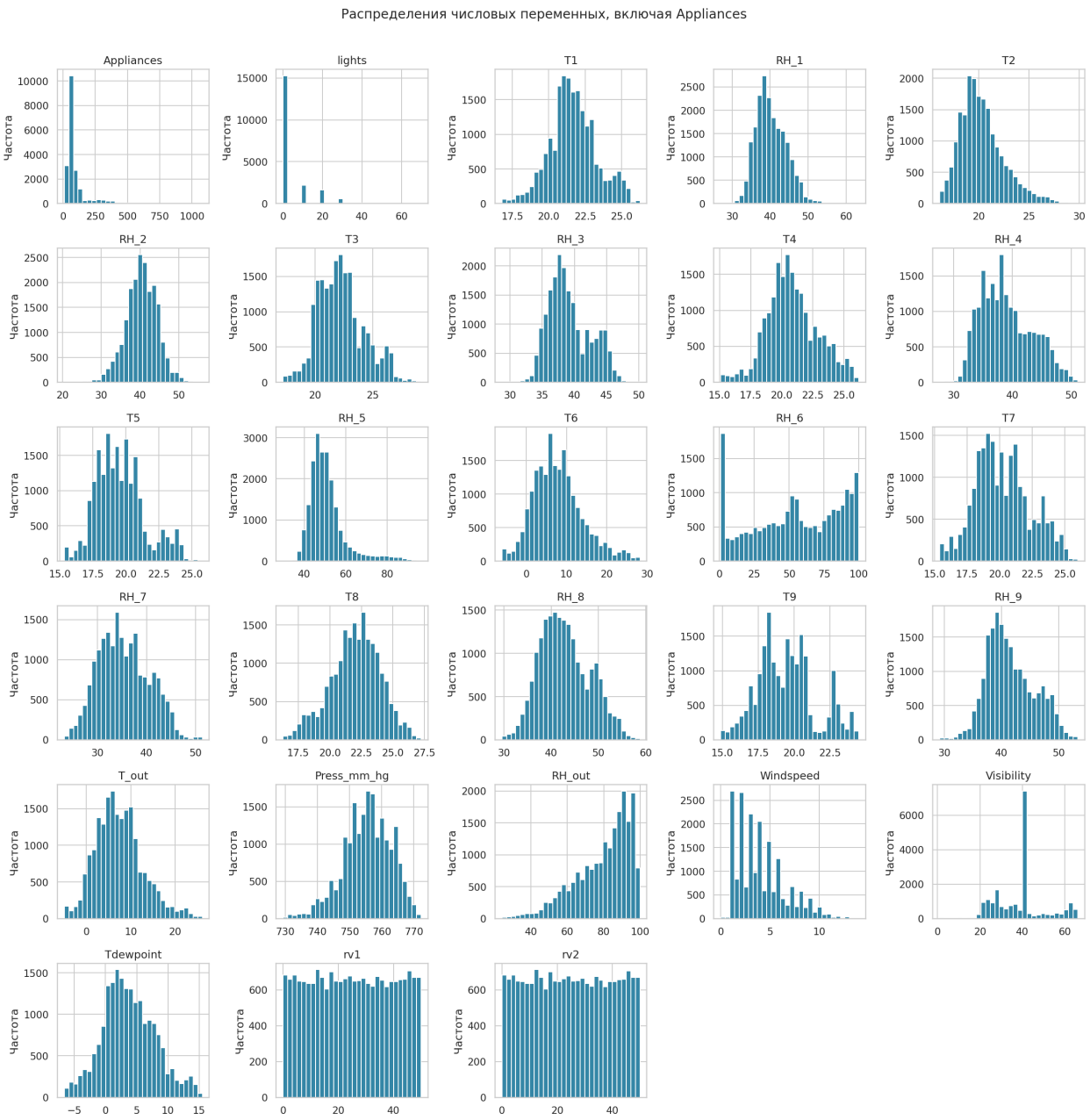


Рис. 1: Распределения числовых переменных

8. Выбросы: как исследованы и почему не удалены

В основном notebook исследовались гистограмма цели, описательные статистики и большие ошибки на графиках остатков. Отдельной фильтрации выбросов, boxplot или IQR-отсечения в сокращённом коде нет. На защите не следует говорить, что такая фильтрация была выполнена.

Дополнительно для защиты применено диагностическое правило Тьюки:

```
q1, q3 = y.quantile([0.25, 0.75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outlier_mask = (y < lower) | (y > upper)
print(lower, upper, outlier_mask.sum())
```

Q1 = 50, Q3 = 100, IQR = 50 Wh. Границы: **−25 и 175 Wh**. Выше верхней границы находятся **2138 наблюдений, 10.83% набора**; ниже нижней — ни одного.

Это статистически необычные значения относительно центральной части распределения, а не автоматически ошибочные измерения. Высокий расход может соответствовать реальной работе мощных приборов. Поэтому пики сохранены: их удаление изменило бы задачу и искусственно уменьшило RMSE. Удаление потребовало бы предметных доказательств неисправности измерений и правила, определённого без подгонки по тестовой метрике.

9. Удаление пропусков и оставшееся число наблюдений

Основной код: lab1.py, строки 70–70.

```
clean = raw.dropna().drop_duplicates().copy()
```

dropna() без дополнительных параметров удаляет строки, в которых есть хотя бы одно пропущенное значение. drop_duplicates() затем оставляет первое появление каждой полной строки. copy() создаёт отдельную таблицу для дальнейшей работы.

В этой копии данных обе операции ничего не удаляют: было и осталось **19 735 наблюдений**. Заполнение средним или медианой не выполнялось: пропусков не было. После последующего исключения четырёх столбцов формируются X с 25 столбцами и y.

10. Категориальные признаки и их кодирование

В матрице X **нет категориальных предикторов**: все 25 столбцов числовые. date после загрузки имеет тип datetime, затем исключается. Appliances — числовая цель; lights с небольшим числом возможных уровней остаётся количественной энергией, а не кодом категории.

Поэтому One-Hot Encoding, LabelEncoder и порядковое кодирование в коде не используются. Требование задания о категориях условное, и для выбранного набора необходимость в кодировании отсутствует.

Если бы появился неупорядоченный признак вроде «тип комнаты», обычно подошёл бы One-Hot Encoding: значения не получили бы искусственных расстояний 1–2–3. При OLS со свободным членом одну базовую категорию можно убрать через drop="first", чтобы избежать точной зависимости между индикаторами. Это пример возможного расширения, а не выполненный этап нашей работы.

11. Какие признаки масштабируются и где

Основной код: lab1.py, строки 99–102.

```
steps = [("scale", StandardScaler())]
if use_pca:
    steps.append(("pca", PCA(n_components=.95, svd_solver="full")))
pipe = Pipeline(steps + [("reg", regressor)])
```

StandardScaler масштабирует **все 25 столбцов X** до регрессии. В PCA-ветке сначала выполняется такое же масштабирование, затем PCA. Ни Appliances, ни

date, ни rv1/rv2 скейлер не получают.

По умолчанию StandardScaler вычитает среднее и делит на стандартное отклонение: $z_j = (x_j - \mu_j)/s_j$. Среднее и масштаб оцениваются по данным обучения. Используется дисперсия с ddof=0; это небольшое техническое отличие от стандартного отклонения, выводимого pandas.describe() с ddof=1.

Стандартизация делает штрафы L1/L2 сопоставимыми для разных единиц измерения и не позволяет давлению доминировать в PCA только из-за большого численного масштаба. Она не делает распределения нормальными и не устраняет корреляции. Для OLS с полным набором признаков и свободным членом обратимое изменение масштаба само по себе не меняет пространство возможных прогнозов.

12. Различие fit, transform и fit_transform

fit оценивает параметры преобразования. Для StandardScaler это mean_ и scale_; для PCA — mean_, components_ и explained_variance_ratio_. Для регрессии fit обучает коэффициенты и свободный член.

transform применяет уже найденные параметры к новым данным и не должен переоценивать их по этим данным. **fit_transform** совмещает обучение преобразования и получение преобразованного обучающего массива.

Дополнительный наглядный пример, эквивалентный логике Pipeline:

```
scaler = StandardScaler()
scaler.fit(X_train)
Z_train = scaler.transform(X_train)
Z_test = scaler.transform(X_test)

# Для обучающей части первые две операции можно объединить:
scaler = StandardScaler()
Z_train = scaler.fit_transform(X_train)
Z_test = scaler.transform(X_test)
```

В диагностике PCA основной код использует StandardScaler().fit_transform(X_train). При predict у Pipeline к тесту последовательно применяются transform, затем reg.predict; fit на тесте не вызывается.

13. На каких данных обучается предобработка и как исключён тест

Основной код: lab1.py, строки 103–109.

```
search = GridSearchCV(pipe, grid, cv=inner_cv, scoring="neg_root_mean_squared_error",
                      n_jobs=1, error_score="raise")
cv = cross_validate(search, X_train, y_train, cv=outer_cv, scoring=scoring,
                    n_jobs=1, error_score="raise")
search.fit(X_train, y_train)
fitted[name] = search.best_estimator_
pred = fitted[name].predict(X_test)
```

Финальный search.fit получает только X_train и y_train. Итоговые модели никогда не передают X_test в fit. X_test впервые нужен для predict, после чего y_test используется для оценки ошибки.

Во вложенной CV scaler и PCA дополнительно обучаются **только на обучающей части соответствующего фолда**, поскольку находятся внутри Pipeline, который оборачивается в GridSearchCV и cross_validate. Это исключает использование проверочного фолда при вычислении средних, масштабов и направлений PCA.

Диагностические VIF и PCA также используют X_train. При этом описательные гистограммы и корреляционная матрица в разделе 2 построены по всему raw. Это следует честно отличать от обучения: по ним не подбирались alpha, скейлер, PCA и

набор реальных предикторов. Для наиболее строгого протокола разведочный анализ тоже лучше ограничивать train; в текущем коде он остаётся описанием исходного набора.

14. Код корреляционной матрицы и вошедшие столбцы

Основной код: lab1.py, строки 62–68.

```
corr = numeric.corr()
plt.figure(figsize=(14, 12))
sns.heatmap(corr, cmap="RdBu_r", vmin=-1, vmax=1, center=0, square=True)
plt.title("Корреляции Пирсона: исходный набор")
show_plot("02_correlations.png")
target_corr = corr["Appliances"].drop("Appliances").sort_values(key=abs, ascending=False)
display(target_corr.head(6).to_frame("Корреляция с Appliances"))
```

numeric ранее сформирован через select_dtypes(include=np.number). В матрице **28 × 28** присутствуют Appliances, lights, девять температур T1–T9, девять влажностей RH_1–RH_9, шесть погодных показателей T_out, Press_mm_hg, RH_out, Windspeed, Visibility, Tdewpoint и rv1/rv2. date отсутствует.

Цель включена в описательную матрицу, чтобы видеть её связи с предикторами. Это не означает, что она попадает в X: при подготовке Appliances исключается. В расчёте VIF цель также отсутствует. Красный цвет соответствует положительной корреляции, синий — отрицательной; шкала от −1 до 1. Последние две строки выводят шесть максимальных по модулю связей с целью.

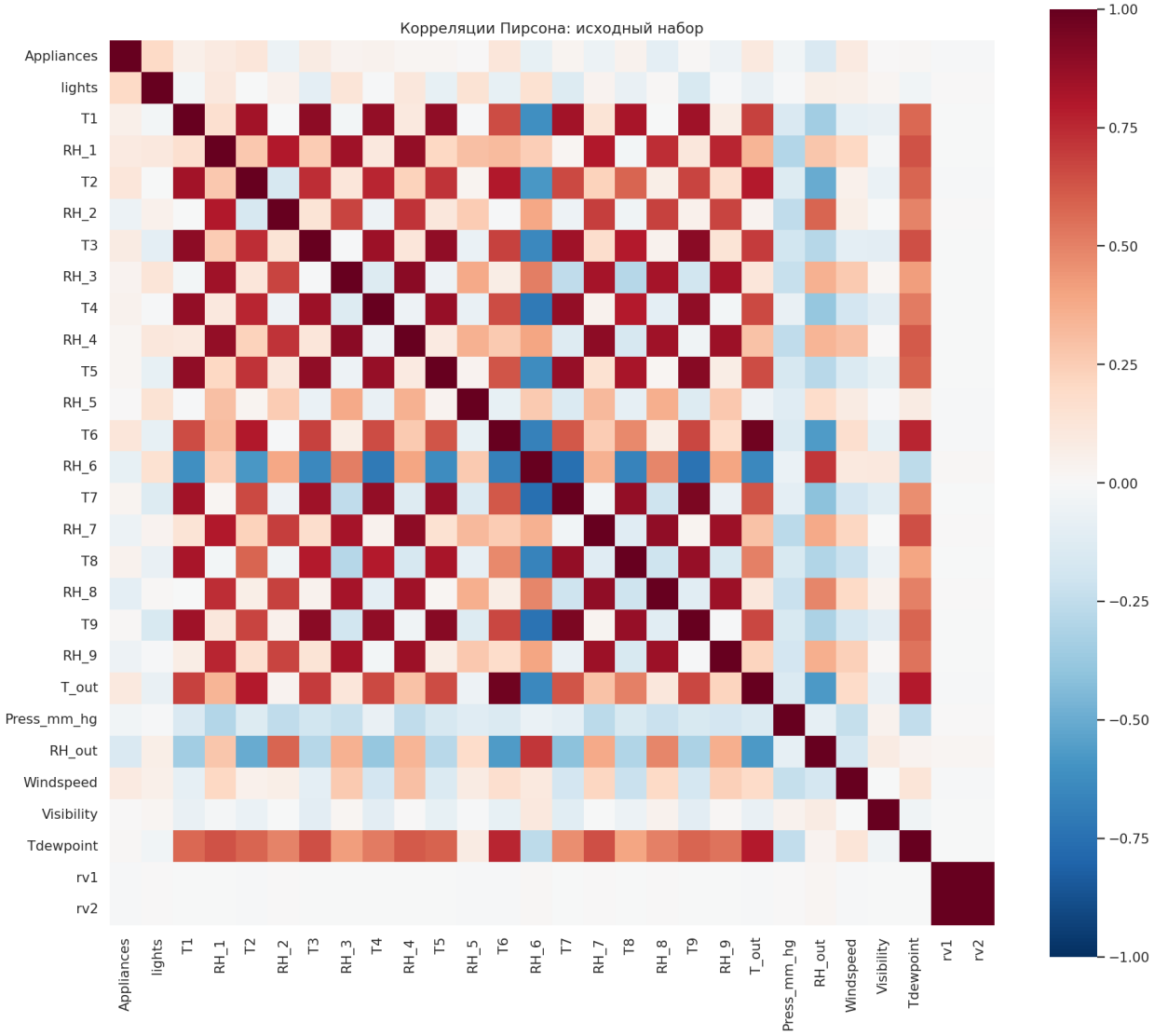


Рис. 2: Корреляционная матрица исходного набора

15. Метод корреляции и причина выбора

У pandas.DataFrame.corr() по умолчанию используется **корреляция Пирсона**, то есть линейная связь:

$$r_{xy} = \frac{\operatorname{cov}(x, y)}{s_x s_y}.$$

Выбор согласован с задачей линейной регрессии: важно изучить линейные зависимости предикторов. Коэффициент безразмерен и не меняется от положительного линейного масштабирования столбцов, поэтому предварительный StandardScaler для этой матрицы не требуется.

Пирсон чувствителен к сильным выбросам и может не обнаружить нелинейную связь, например симметричную квадратичную зависимость. Нулевой коэффициент не доказывает независимость. Для монотонных нелинейных зависимостей можно дополнительно использовать Спирмена, но в этой работе он не рассчитывался. Высокая корреляция сама по себе не доказывает причинного влияния.

16. Пары с наиболее сильной связью

Самая сильная связь во всей исходной таблице — **rv1 и rv2: r = 1**, поскольку столбцы идентичны. Среди 25 оставшихся физических предикторов:

Пара	r Пирсона
T6 — T_out	0.974787
T7 — T9	0.944776
T5 — T9	0.911055
T3 — T9	0.901324
RH_3 — RH_4	0.898978
RH_4 — RH_7	0.894301
T1 — T3	0.892402
T4 — T9	0.889439

T6 и T_out обе описывают наружную температуру, хотя источники измерения различаются. Высокая связь между комнатными температурами отражает общий температурный режим дома; влажности тоже изменяются совместно. На тепловой карте это видно как насыщенные блоки вне главной диагонали.

Связи с целью намного слабее: максимальная по модулю — lights, r ≈ **0.197278**; RH_out имеет r ≈ **−0.152282**. Сильная корреляция предикторов между собой не означает сильной зависимости Appliances от каждого из них.

17. VIF: код, входные данные и интерпретация

Основной код: lab1.py, строки 78–84.

```
def vif(frame):
    matrix = add_constant(StandardScaler().fit_transform(frame))
    return pd.Series([variance_inflation_factor(matrix, i + 1) for i in range(frame.shape[1])],
                     index=frame.columns, name="VIF")

vif_before = vif(X_train)
display(vif_before.sort_values(ascending=False).round(2).to_frame())
```

Передаётся X_train: **15 788 × 25**, без цели, даты и случайных контролей. StandardScaler центрирует и масштабирует предикторы; add_constant добавляет свободный член во вспомогательные регрессии. После добавления константы исходный i-й предиктор находится в столбце i+1, поэтому используется i+1. Саму константу в таблицу VIF не выводим.

$$VIF_j = \frac{1}{1 - R_j^2},$$

где R_j^2 получен регрессией j-го предиктора на остальные. VIF около 1 означает отсутствие выраженной линейной предсказуемости по остальным столбцам; >5 — предупреждение, >10 — сильную зависимость. Это ориентиры, а не универсальное правило удаления.

17 из 25 признаков имеют VIF >10. Максимум — T_out ≈150.12; Tdewpoint ≈88.10, RH_out ≈49.90, T6 ≈33.75. Стандартизация не уменьшает саму мультиколлинеарность: VIF не зависит от единиц измерения признака. Высокий VIF описывает геометрию X, а не качество модели по цели.

18. Что предложить удалить или преобразовать

Выполнено: исключены date и два искусственных случайных контроля; для устранения зависимостей между физическими предикторами применён PCA. Исходные три регрессии сохраняют все 25 физических столбцов, чтобы сравнение с PCA было содержательным.

Возможный альтернативный вариант для интерпретируемой модели: оставить один из T6/T_out, затем повторно оценить VIF. Нужно также исследовать совместное использование T_out, RH_out и Tdewpoint: эти погодные величины связаны физически и дают большие VIF. В группе комнатных температур можно оставить репрезентативные датчики или объединить информацию.

Однако нельзя автоматически удалить все 17 признаков с VIF >10: после удаления одного зависимости остальных меняются, а полезная для цели информация может исчезнуть. Правило выбирается на train с CV, VIF пересчитывается после изменения набора. Тест для этого не используется. Низкая парная корреляция с y также не доказывает бесполезность признака в многомерной модели.

19. Разделение на train/test и размеры выборок

Основной код: lab1.py, строки 74–76.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.20, random_state=42)
display(pd.DataFrame({"Выборка": ["Train", "Test"],
                      "Строк": [len(X_train), len(X_test)], "Признаков": X.shape[1]}))
```

test_size=.20 выделяет 20% наблюдений под тест. shuffle по умолчанию True: это случайное, а не хронологическое разбиение. X и y передаются совместно, поэтому строки предикторов и ответы остаются согласованы.

Массив	Размер
X_train	15 788 × 25
X_test	3947 × 25
y_train	15 788
y_test	3947

Отдельная постоянная валидационная таблица не создаётся: выбор alpha и CV-оценивание происходят внутри тренировочной части. Тестовая часть не пересоздаётся при переходе от исходных признаков к PCA.

20. Назначение random_state и последствия его изменения

random_state задаёт воспроизводимое состояние генератора при случайном перемешивании. В train_test_split и внешнем KFold используется **42**, во внутреннем KFold — **17**. Это фиксирует состав выборок и фолдов при повторении запуска с теми же данными и версиями библиотек.

При другом значении в train/test могут попасть другие наблюдения, изменятся train-статистики, направления PCA, оценки CV, выбранный alpha и тестовые метрики. Общие размеры выборок при test_size=.20 сохранятся. Само число 42 не улучшает качество и не имеет предметного смысла. Перебирать seeds ради красивого тестового результата нельзя считать корректным подбором модели.

PCA с full SVD и Ridge с solver="svd" детерминированы. Lasso использует циклический coordinate descent, а не случайный порядок признаков, поэтому её собственный random_state здесь не является источником случайности.

21. Подходит ли случайное разбиение для этих данных

В наборе есть явный временной порядок: **2016-01-11 17:00 — 2016-05-27 18:00**, шаг **10 минут**. Это повторные измерения **одного дома**, а не независимые случайно выбранные дома. Близкие по времени строки могут быть похожи.

Случайное разбиение допускается учебной постановкой и позволяет сравнить методы при одинаковых условиях. Оно оценивает восстановление расхода в пределах наблюдавшегося периода при известных текущих признаках. Для утверждения о прогнозировании будущих периодов такое разбиение недостаточно: соседние моменты оказываются в train и test, а будущие наблюдения могут участвовать в обучении относительно отдельных тестовых моментов.

Возможное изменение протокола, не выполненное в текущей версии:

```
from sklearn.model_selection import TimeSeriesSplit

X_train_time, X_test_time, y_train_time, y_test_time = train_test_split(
    X, y, test_size=0.20, shuffle=False
)
time_cv = TimeSeriesSplit(n_splits=5, gap=144)
```

Здесь исходная таблица уже упорядочена по date; в общем случае её нужно сначала отсортировать. gap=144 означает суточный разрыв при 10-минутном шаге, но его достаточность надо исследовать. Кроме того, для настоящего прогноза заранее нужно обеспечить доступность будущих признаков: текущие lights и погода не становятся известны заранее автоматически.

22. Создание и обучение трёх регрессий

Основной код: lab1.py, строки 91–108.

```
def assess_family(use_pca=False):
    candidates = {
        "OLS": (LinearRegression(), {}),
        "Lasso": (Lasso(max_iter=50000), {"reg__alpha": np.logspace(-2, 2, 9)}),
        "Ridge": (Ridge(solver="svd"), {"reg__alpha": np.logspace(-3, 5, 17)}),
    }
    rows, fitted = [], {}
    for name, (regressor, grid) in candidates.items():
        steps = [("scale", StandardScaler())]
        if use_pca:
            steps.append(("pca", PCA(n_components=.95, svd_solver="full")))
        pipe = Pipeline(steps + [("reg", regressor)])
        search = GridSearchCV(pipe, grid, cv=inner_cv, scoring="neg_root_mean_squared_error",
                              n_jobs=1, error_score="raise")
        cv = cross_validate(search, X_train, y_train, cv=outer_cv, scoring=scoring,
                            n_jobs=1, error_score="raise")
        search.fit(X_train, y_train)
        fitted[name] = search.best_estimator_
```

LinearRegression — обычная линейная регрессия, Lasso — L1-регуляризация, Ridge — L2-регуляризация. Регрессор становится последним шагом Pipeline. Явного regressor.fit в основной функции нет: его вызывает fit конвейера, запускаемый GridSearchCV и cross_validate.

Первое семейство создаётся вызовом **base_metrics, base_models = assess_family()**. Результат search.best_estimator_ — уже обученный Pipeline с выбранным alpha, сохранённый под именем модели. Для OLS сетка параметров пустая: выбора alpha нет, хотя общая обёртка GridSearchCV сохраняется.

23. Явные параметры моделей и параметры по умолчанию

Версия scikit-learn — **1.8.0**. В конструкторах явно заданы:

Объект	Явно задано	Зачем
LinearRegression	Специальных параметров нет	Обычный OLS со стандартными настройками
Lasso	max_iter=50000	Достаточный предел итераций при коррелированных признаках
Ridge	solver="svd"	Устойчивое решение плотной задачи при корреляциях
PCA в модели	n_components=.95, svd_solver="full"	Порог дисперсии и полный SVD
GridSearchCV	cv=inner_cv, scoring=neg RMSE, n_jobs=1, error_score="raise"	Подбор по RMSE и явная ошибка при неудаче

Параметр alpha у Lasso/Ridge затем **явно задаётся поиском по сетке**; оставшийся в конструкторе начальный default=1.0 не означает, что итоговые модели обучены с alpha=1.

Для всех регрессий оставлено fit_intercept=True и positive=False. Поэтому оценивается свободный член и нет ограничения неотрицательности коэффициентов или прогнозов. У Lasso: tol=0.0001, selection="cyclic", warm_start=False. У Ridge: tol=0.0001; для SVD этот параметр не является итерационным критерием сходимости. У OLS есть default tol=1e-6, но для плотного решения через least squares он не играет роль, аналогичную Lasso. У StandardScaler: with_mean=True, with_std=True. У PCA: whiten=False.

Дополнительно для просмотра фактических настроек:

```
print(base_models["Lasso"]["reg"].get_params())
print(base_models["Ridge"]["reg"].get_params())
print(pca_models["OLS"]["pca"].get_params())
```

Предупреждение ConvergenceWarning превращено в ошибку. В проверенном запуске Lasso сошлась за **897** итераций на исходных признаках и за **2** после PCA, что меньше 50 000.

24. Подбор alpha: сетки и полученные значения

Основной код: lab1.py, строки 92–95.

```
candidates = {
    "OLS": (LinearRegression(), {}),
    "Lasso": (Lasso(max_iter=50000), {"reg__alpha": np.logspace(-2, 2, 9)}),
    "Ridge": (Ridge(solver="svd"), {"reg__alpha": np.logspace(-3, 5, 17)}),
```

np.logspace создаёт равномерную сетку **в логарифмической шкале**: разница порядков важнее одинакового арифметического шага.

Lasso, 9 значений:

```
[1.000000e-02 3.162278e-02 1.000000e-01 3.162278e-01 1.000000e+00 3.162278e+00
 1.000000e+01 3.162278e+01 1.000000e+02]
```

Ridge, 17 значений:

```
[1.000000e-03 3.162278e-03 1.000000e-02 3.162278e-02 1.000000e-01 3.162278e-01
 1.000000e+00 3.162278e+00 1.000000e+01 3.162278e+01 1.000000e+02 3.162278e+02
 1.000000e+03 3.162278e+03 1.000000e+04 3.162278e+04 1.000000e+05]
```

GridSearchCV проверяет каждый alpha на трёх внутренних фолдах. Выбирается максимум neg_root_mean_squared_error, что равносильно минимуму RMSE. Затем лучший конвейер переобучается на всём доступном ему train.

Пространство	alpha Lasso	alpha Ridge
25 исходных признаков	0.01	3.1622776602
11 компонент PCA	0.1	31.6227766017

У OLS alpha отсутствует; NaN в таблице означает «не применяется», а не пропуск в обучающих данных. Для Lasso на исходных признаках выбран нижний край сетки: можно сказать, что среди проверенных значений предпочтителен слабый штраф. Глобальный оптимум по всем alpha не доказан. Сравнивать число alpha между Lasso и Ridge напрямую нельзя: у функций потерь разная нормировка.

25. Кросс-валидация: код, число фолдов и метрика

Основной код: lab1.py, строки 86–89.

```
outer_cv = KFold(n_splits=5, shuffle=True, random_state=42)
inner_cv = KFold(n_splits=3, shuffle=True, random_state=17)
scoring = {"RMSE": "neg_root_mean_squared_error", "R2": "r2",
           "MAPE": "neg_mean_absolute_percentage_error"}
```

Основной код: lab1.py, строки 103–107.

```
search = GridSearchCV(pipe, grid, cv=inner_cv, scoring="neg_root_mean_squared_error",
                      n_jobs=1, error_score="raise")
cv = cross_validate(search, X_train, y_train, cv=outer_cv, scoring=scoring,
                    n_jobs=1, error_score="raise")
search.fit(X_train, y_train)
```

Внешний KFold содержит **5 фолдов**. Каждое наблюдение исходного train один раз становится внешним проверочным; на остальных фолдах обучаются модель и внутренний подбор параметров. Внутренний KFold содержит **3 фолда** и выбирает alpha. Это вложенная CV 5×3, а не одна обычная 15-fold CV. После внешней оценки search.fit ещё раз подбирает alpha на полном train.

Для настройки используется только **RMSE**. Для независимой внешней оценки собираются RMSE, R^2 и MAPE. Ошибки в интерфейсе sklearn имеют знак минус, поскольку поиск максимизирует score. При выводе RMSE и MAPE знак восстанавливается; R^2 не меняется.

В таблице CV RMSE — среднее пяти RMSE, а CV σ RMSE — их стандартное отклонение с ddof=1. Это разброс между фолдами, не доверительный интервал. best_score_ внутреннего поиска не выдаётся за независимую внешнюю оценку.

26. Где обучаются scaler и PCA во время CV

Основной код: lab1.py, строки 99–106.

```
steps = [("scale", StandardScaler())]
if use_pca:
    steps.append(("pca", PCA(n_components=.95, svd_solver="full")))
pipe = Pipeline(steps + [("reg", regressor)])
search = GridSearchCV(pipe, grid, cv=inner_cv, scoring="neg_root_mean_squared_error",
                      n_jobs=1, error_score="raise")
cv = cross_validate(search, X_train, y_train, cv=outer_cv, scoring=scoring,
                    n_jobs=1, error_score="raise")
```

Оба преобразования входят в **тот же Pipeline, который получает GridSearchCV**. Следовательно, при проверке каждого alpha и каждого внутреннего фолда scaler.fit и PCA.fit заново используют только внутреннюю обучающую часть. Внешний cross_validate также получает целый поиск, поэтому внешний проверочный фолд не участвует в настройке.

После выбора alpha Pipeline заново обучается на всей обучающей части соответствующего внешнего фолда; после завершения CV — на полном X_train. Параметр refit у GridSearchCV по умолчанию True.

Отдельный массив scaled, рассчитанный в диагностическом разделе, **не передаётся** в assess_family. Он нужен только для спектра PCA и VIF. Если бы PCA заранее обучили на всём X_train, а потом делали CV на готовых компонентах, проверочные фолды уже повлияли бы на преобразование. Такого предварительного преобразования входа CV в нашей функции нет.

27. Pipeline: последовательность и назначение шагов

Основной код: lab1.py, строки 99–102.

```
steps = [("scale", StandardScaler())]
if use_pca:
    steps.append(("pca", PCA(n_components=.95, svd_solver="full")))
pipe = Pipeline(steps + [("reg", regressor)])
```

Для исходных моделей цепочка:

25 исходных столбцов → scale: StandardScaler → reg: OLS/Lasso/Ridge.

Для моделей с PCA:

25 исходных столбцов → scale: StandardScaler → pca: PCA(0.95) → reg.

При fit промежуточные преобразования обучаются и преобразуют train, последний шаг обучается по преобразованному X и y. При predict применяются только уже обученные преобразования, затем регрессор. Это обеспечивает одинаковые правила работы с train и test и корректную CV.

Имена шагов используются для доступа, например pipe[“reg”].coef_, и для поиска гиперпараметров: reg__alpha означает параметр alpha шага reg. Полный train даёт 11 компонент. Во фолдах число компонент определяется заново по тем же 95%; фиксировать его по внешнему проверочному фолду нельзя.

28. Коэффициенты обученных моделей и их различия

В сокращённом notebook отдельная таблица коэффициентов не печатается. Ниже приведены **дополнительно рассчитанные значения именно его финальных моделей**. Их можно получить после выполнения ячеек:

```
coefs = pd.DataFrame(
    {name: model["reg"].coef_ for name, model in base_models.items()},
    index=X_train.columns,
)
display(coefs)
```

```
print({name: model["reg"].intercept_ for name, model in base_models.items()})
```

Коэффициенты при **стандартизированных** исходных признаках:

Признак	OLS	Lasso	Ridge
lights	15.370254	15.376035	15.389529
T1	0.119070	-0.015426	-0.224109
RH_1	62.939727	62.748903	62.566721
T2	-38.645268	-38.122192	-37.896808
RH_2	-55.397637	-54.911091	-54.752770
T3	51.936510	51.733543	51.835277
RH_3	14.554066	14.352331	14.488084
T4	-7.955635	-7.914604	-8.003463
RH_4	-1.764067	-1.591306	-1.776526
T5	-1.605908	-1.782370	-1.771255
RH_5	1.911667	1.907050	1.918048
T6	45.075244	44.152110	44.270033
RH_6	10.655859	10.227836	10.421000
T7	4.744912	4.442669	4.669705
RH_7	-8.496606	-8.421020	-8.501712
T8	14.594692	14.526961	14.560335
RH_8	-23.800470	-23.880679	-23.806201
T9	-30.663830	-30.321735	-30.484792
RH_9	-4.280957	-4.251725	-4.314611
T_out	-55.947096	-52.176903	-53.316953
Press_mm_hg	1.196001	1.166501	1.190292
RH_out	-15.976640	-14.047639	-14.737543
Windspeed	4.330849	4.377486	4.375530
Visibility	1.910268	1.888114	1.900299
Tdewpoint	21.042191	18.348300	19.329384

Свободный член всех трёх исходных моделей — **97.949709 Wh**. Это среднее y_train: все стандартизированные признаки центрированы. Коэффициент выражает изменение прогнозируемых Wh при увеличении признака на одно его train-стандартное отклонение при неизменных остальных входах. Например, для OLS рост RH_1 на одно стандартное отклонение соответствует изменению прогноза примерно на +62.94 Wh при прочих равных.

Регуляризация уменьшила общую норму коэффициентов: L2-нормы примерно 139.90 для OLS, 136.90 для Lasso, 137.49 для Ridge. При этом отдельный коэффициент не обязан уменьшаться по модулю: из-за зависимостей предикторов веса перераспределяются. У T1 знак меняется с +0.119 в OLS на −0.015 в Lasso и −0.224 в Ridge при близких прогнозах. Это пример нестабильности малых условных эффектов.

У выбранной Lasso нет ни одного точно нулевого коэффициента.
L1-регуляризация умеет занулять веса, но не обязана делать это при каждом alpha. Здесь выбран слабый штраф 0.01; нельзя говорить, что модель фактически отобрала меньше 25 признаков.

После PCA коэффициенты относятся к компонентам:

Компонента	OLS	Lasso	Ridge
PC1	2.411073	2.400369	2.410556

Компонента	OLS	Lasso	Ridge
PC2	-0.745813	-0.731632	-0.745601
PC3	11.536058	11.480823	11.523309
PC4	17.582232	17.493699	17.551109
PC5	3.331131	3.235994	3.324795
PC6	2.742953	2.640328	2.737327
PC7	-11.660902	-11.542599	-11.633336
PC8	1.147288	0.968790	1.143200
PC9	-6.691987	-6.503225	-6.666781
PC10	-21.353438	-21.096257	-21.244005
PC11	-23.909055	-23.497913	-23.713772

Их свободный член также ≈97.949709 Wh. Компоненты не имеют единиц исходных датчиков и при whiten=False не приведены к единичной дисперсии. Знак и величину такого коэффициента нельзя напрямую назвать влиянием конкретной температуры.

Для интерпретации исходных моделей в первоначальных единицах:

```
pipe = base_models["OLS"]
beta_original = pipe["reg"].coef_ / pipe["scale"].scale_
intercept_original = (
    pipe["reg"].intercept_
    - np.dot(beta_original, pipe["scale"].mean_)
)
```

Например, OLS в исходных единицах имеет коэффициент lights ≈1.930226 Wh прогноза на +1 Wh освещения; RH_1 ≈15.905544 Wh на +1 процентный пункт. Свободный член этой формы — 23.193744 Wh. Это математическая условная интерпретация, не доказательство причинного влияния; высокая мультиколлинеарность особенно мешает трактовать отдельные знаки.

29. Получение прогнозов и выборка итоговой оценки

Основной код: lab1.py, строки 107–109.

```
search.fit(X_train, y_train)
fitted[name] = search.best_estimator_
pred = fitted[name].predict(X_test)
```

fitted[name] — финальный обученный Pipeline, refit на полном X_train. Его predict(X_test) масштабирует тест сохранённым scaler, при необходимости проецирует его обученным PCA и вызывает reg.predict.

Итоговые метрики таблицы RMSE, Wh; R^2; MAPE, % рассчитаны по **3947 отложенным тестовым наблюдениям**. CV-метрики получены отдельно на внешних фолдах тренировочной части. Сама y_test в predict не передаётся: она требуется только при вычислении ошибок. Прогнозы всех шести моделей с реальными ответами сохранены в results/predictions.csv. Они не обрезались до нуля и не проходили обратное логарифмирование: цель обучалась в исходной шкале Wh.

30. Где вычисляются RMSE, R^2 и MAPE и какие массивы используются

Основной код: lab1.py, строки 117–118.

```
"RMSE, Wh": root_mean_squared_error(y_test, pred), "R²": r2_score(y_test, pred),
"MAPE, %": 100 * mean_absolute_percentage_error(y_test, pred),
```

Первый аргумент — **y_test**, реальные 3947 значений цели. Второй — **pred**, 3947 прогнозов текущей модели для X_test. Порядок элементов согласован, поскольку X и y совместно разделены train_test_split и predict сохраняет порядок строк входа.

Для RMSE вычисляется корень из среднего квадрата ошибок. R^2 сравнивает сумму квадратов ошибок со суммой квадратов отклонений ответов от среднего **на оцениваемой выборке**. MAPE усредняет абсолютные относительные ошибки и затем переводится в проценты множителем 100.

Для CV эти же типы показателей считаются встроенными scorers на внешних проверочных фолдах. Финальные тестовые значения не подменяются ошибкой обучения или лучшим внутренним CV-score.

31. Как получить RMSE через MSE

В действующей программе используется **root_mean_squared_error**, поэтому дополнительного np.sqrt в строке метрики нет. Эквивалентное вычисление:

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)

# Или непосредственно по массивам:
rmse_manual = np.sqrt(np.mean((y_test.to_numpy() - pred) ** 2))
```

$$MSE = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2, \qquad RMSE = \sqrt{MSE}.$$

MSE имеет размерность Wh^2; после извлечения корня получается Wh. Нельзя взять sqrt от MAE или от R^2 и назвать результат RMSE. Приведённая дополнительная формула относится к тому же одномерному регрессионному ответу без весов наблюдений.

32. Единицы RMSE, R^2 и MAPE

RMSE выводится в Wh (Вт·ч) — тех же единицах, что Appliances. Это не W, не kWh и не процент. Никакого деления на 1000 в коде нет. R^2 — безразмерное число: например, 0.169442.

sklearn.mean_absolute_percentage_error возвращает **долю**, а не уже готовый процент. Поэтому код умножает её на 100: 0.6288465 превращается в 62.88465%.

Для CV используется отрицательный scorer MAPE, поэтому в строке CV MAPE применяется множитель **−100**, одновременно меняющий знак и единицы. Тестовой положительной функции нужен только **100**. Дважды умножать на 100 нельзя. Столбец R^2 в таблице не умножен на 100; словесная интерпретация 16.94% объяснённой вариации — отдельная запись того же коэффициента 0.1694.

33. Нулевые и малые значения цели и интерпретация MAPE

В цели **нет нулей**. Минимум — **10 Wh**; таких наблюдений **9**. Значений не выше 20 Wh — **352**. Это небольшие величины относительно редких пиков до 1080 Wh. Перед обучением это явно проверяется: **Основной код: lab1.py, строки 73–73.**

```
assert y.gt(0).all()
```

При y=10 Wh и прогнозе 60 Wh абсолютная ошибка 50 Wh превращается в относительную ошибку **500%**. При y=500 и прогнозе 550 та же абсолютная ошибка

составляет **10%**. Поэтому небольшое потребление получает в MAPE большой относительный вес.

В sklearn деление на ноль технически защищено машинным epsilon, но при y=0 результат может стать практически бессмысленно огромным. В нашей работе эта защита не определяет метрику, поскольку все y>0. MAPE ≈62.88% нужно обсуждать вместе с RMSE и формой распределения цели. Это не «точность классификации 37.12%» и не вероятность правильного ответа.

34. Качество на train и test: есть ли переобучение

В основной таблице отдельно не выводится ошибка обучения. Для защиты она дополнительно рассчитана для уже выбранных финальных моделей:

```
for name, pipe in base_models.items():
    train_prediction = pipe.predict(X_train)
    test_prediction = pipe.predict(X_test)
    print(name,
          root_mean_squared_error(y_train, train_prediction),
          root_mean_squared_error(y_test, test_prediction))
```

Базис	Модель	RMSE train	RMSE test	R^2 train	R^2 test	MAPE train/test, %
Исходные	OLS	94.329	91.172	0.16345	0.16936	61.11 / 62.92
Исходные	Lasso	94.330	91.167	0.16343	0.16944	61.08 / 62.88
Исходные	Ridge	94.329	91.168	0.16344	0.16943	61.10 / 62.90
PCA	OLS	97.536	93.976	0.10559	0.11748	63.39 / 65.07
PCA	Lasso	97.537	93.973	0.10557	0.11752	63.38 / 65.06
PCA	Ridge	97.537	93.974	0.10559	0.11751	63.38 / 65.07

RMSE в обеих колонках измеряется в Wh. **Выраженного разрыва в сторону худшего test нет:** у исходной Lasso train RMSE≈94.33, test≈91.17. Немного меньшая тестовая ошибка не означает ошибку программы: случайная тестовая часть может оказаться проще. Стандартное отклонение цели здесь около 103.14 Wh на train и 100.05 Wh на test.

По этой диагностике нет основания заявлять сильное переобучение. Но это не доказательство надёжного прогноза будущего: временная зависимость делает случайные части похожими. Невысокий R^2 при близких train/test скорее указывает на ограниченную линейную спецификацию и недостаток информации о режиме приборов. Чтобы строго диагностировать недообучение, полезны кривые обучения и сравнение с более выразительными моделями на корректной валидации; одного низкого R^2 для доказательства недостаточно.

35. Как выбрать лучшую модель при несогласии метрик

Критерий выбора установлен заранее — **минимум внешнего CV RMSE на train**. Тест используется для итоговой оценки, а не для смены критерия после просмотра удобного результата.

В основном коде сравнение хранится в comparison; отдельная переменная best_model не создаётся. В тексте отчёта лучший вариант определён по минимальному CV RMSE. Это можно сделать явно дополнительной командой:

```
best = comparison.loc[comparison["CV RMSE"].idxmin()]
print(best[["Признаки", "Модель", "CV RMSE", "RMSE, Wh", "R²", "MAPE, %"]])
```

Получается **Lasso / Исходные**, CV RMSE≈94.52894 Wh. При выборе по CV MAPE минимальным стал бы Ridge / Исходные: ≈61.21292% против ≈61.21797% у Lasso. Разница очень мала. Таким образом, конфликт критериев возможен уже в фактических результатах. Он разрешается назначением основной метрики по задаче, а не попыткой доказать, что одна модель обязана победить по всему.

Отличия исходных OLS/Lasso/Ridge по RMSE тысячные доли Wh при разбросе внешней CV около 1.3 Wh. Формальный минимум не является доказательством практически или статистически значимого превосходства.

36. Стандартизация перед PCA и выборка её обучения

Основной код: lab1.py, строки 143–143.

```
scaled = StandardScaler().fit_transform(X_train)
```

Это диагностическое преобразование **X_train**, без X_test и без y. scaled имеет размер 15 788 × 25. Для каждого столбца из train вычтено среднее и выполнено деление на его train-масштаб.

В моделях та же операция находится внутри Pipeline, поэтому на CV оценки среднего и масштаба заново вычисляются по текущему обучающему фолду. Отдельный диагностический scaler не заменяет scaler модели и не используется для настройки по проверочным наблюдениям.

PCA сам умеет центрировать вход, но **не заменяет стандартизацию разных физических единиц**. Без StandardScaler высокая численная дисперсия давления или влажности могла бы доминировать над температурами только из-за масштаба измерения.

37. Где создаётся и обучается PCA, что подаётся на вход

Основной код: lab1.py, строки 143–148.

```
scaled = StandardScaler().fit_transform(X_train)
pca_full = PCA(svd_solver="full").fit(scaled)
cumulative = pca_full.explained_variance_ratio_.cumsum()
k95 = int(np.searchsorted(cumulative, .95) + 1)
components = pd.DataFrame(pca_full.transform(scaled)[:, :k95],
                          columns=[f"PC{i+1}" for i in range(k95)])
```

В диагностике **pca_full** обучается на стандартизированном train и сохраняет все 25 направлений. Он нужен для полного спектра и выбора k95. На вход PCA поступают только числа предикторов; Appliances отсутствует.

В оцениваемых регрессиях создаётся другой PCA: **Основной код: lab1.py, строки 99–102.**

```
steps = [("scale", StandardScaler())]
if use_pca:
    steps.append(("pca", PCA(n_components=.95, svd_solver="full")))
pipe = Pipeline(steps + [("reg", regressor)])
```

Он оставляет число компонент по порогу 0.95 и обучается внутри Pipeline. Сама строка конструктора ещё ничего не обучает: обучение запускается при

GridSearchCV.fit и cross_validate. PCA — метод без учителя: его направления зависят от X, а не от y.

38. Правило выбора количества компонент

В основном конвейере задано **n_components=.95**: это вещественный порог доли дисперсии, а не фиксированное число компонент. Диагностический код находит соответствующее количество: **Основной код: lab1.py, строки 145–146.**

```
cumulative = pca_full.explained_variance_ratio_.cumsum()
k95 = int(np.searchsorted(cumulative, .95) + 1)
```

На полном train **10 компонент объясняют 94.7357%**, а **11 — 95.7086%** дисперсии. Поэтому итоговое число — 11. Строка PCA(n_components=11) означала бы фиксированное число; её в оцениваемом конвейере нет.

Порог определён заранее, без выбора по тестовой ошибке. На отдельном CV-фолде вычисляется свой спектр и число направлений для того же порога. Сохраняется доля дисперсии признаков, а не доля R^2 и не гарантированная точность прогнозирования цели.

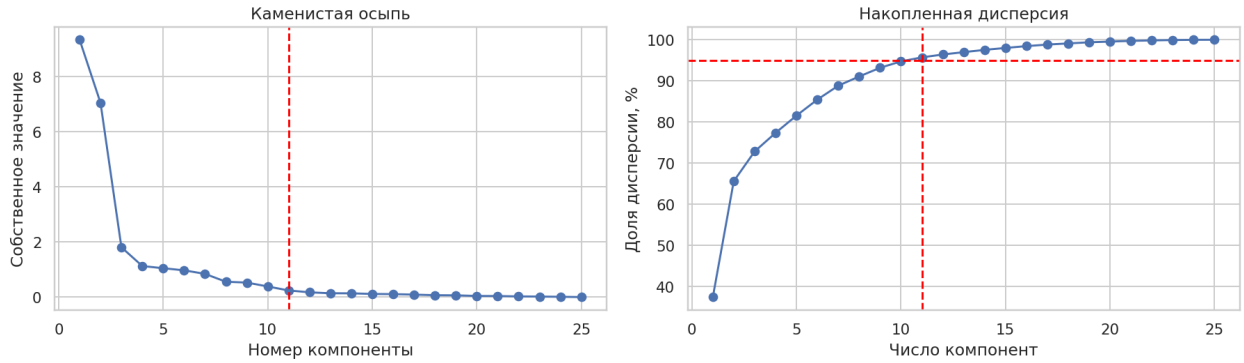


Рис. 3: Каменистая осыпь и накопленная дисперсия

39. explained_variance_ratio_ и накопленная дисперсия

explained_variance_ratio_[j] — доля суммарной дисперсии X, приходящаяся на j-ю компоненту. Если её собственное значение равно λ_j, доля равна $\lambda_j / \sum_l \lambda_l$. В основном коде накопление выполняется так: **Основной код: lab1.py, строки 145–146.**

```
cumulative = pca_full.explained_variance_ratio_.cumsum()
k95 = int(np.searchsorted(cumulative, .95) + 1)
```

Дополнительно для полного вывода:

```
ratio = pca_full.explained_variance_ratio_
display(pd.DataFrame({
    "PC": np.arange(1, len(ratio) + 1),
    "Доля": ratio,
    "Накопленная доля": ratio.cumsum(),
}))
```

PC	Доля 0–1	Доля, %	Накоплено, %
1	0.37371029	37.37103	37.37103
2	0.28206106	28.20611	65.57714
3	0.07241822	7.24182	72.81896

PC	Доля 0–1	Доля, %	Накоплено, %
4	0.04518109	4.51811	77.33707
5	0.04204460	4.20446	81.54153
6	0.03897693	3.89769	85.43922
7	0.03381146	3.38115	88.82037
8	0.02240924	2.24092	91.06129
9	0.02119071	2.11907	93.18036
10	0.01555321	1.55532	94.73568
11	0.00972900	0.97290	95.70858
12	0.00706736	0.70674	96.41532
13	0.00568168	0.56817	96.98349
14	0.00556713	0.55671	97.54020
15	0.00468800	0.46880	98.00900
16	0.00442567	0.44257	98.45157
17	0.00373861	0.37386	98.82543
18	0.00287985	0.28798	99.11341
19	0.00266771	0.26677	99.38018
20	0.00179624	0.17962	99.55981
21	0.00167125	0.16712	99.72693
22	0.00112805	0.11280	99.83974
23	0.00089086	0.08909	99.92882
24	0.00056656	0.05666	99.98548
25	0.00014522	0.01452	100.00000

Первые две компоненты сохраняют около 65.58%, первые три — 72.82%. Для порога 95% этого недостаточно. Доли всех 25 компонент в сумме дают 1 с точностью округления. После усечения сумма долей 11 компонент остаётся 0.957086, а не перенормируется автоматически к 1.

40. Размеры матриц до и после PCA

Этап	Train	Test
Исходные подготовленные X	15 788 × 25	3947 × 25
После StandardScaler	15 788 × 25	3947 × 25
После PCA с порогом 95%	15 788 × 11	3947 × 11

PCA уменьшает **число столбцов**, а не число наблюдений. y_train и y_test не меняются и имеют длины 15 788 и 3947. Размерность сократилась на $(25 - 11)/25 = 56\%$. В диагностическом коде массив components имеет 15 788 строк и 11 столбцов. В регрессионных моделях такие массивы создаются внутри Pipeline. Фолды CV содержат меньше строк и могут иметь своё число компонент, определённое тем же порогом; приведённые размеры относятся к финальному train/test.

41. Преобразование теста в главные компоненты

В основном коде преобразование скрыто внутри вызова pca_models[name].predict(X_test). Для явного показа используется **дополнительный** фрагмент:

```
pipe = pca_models["OLS"]
Z_test = pipe["scale"].transform(X_test)
PC_test = pipe["pca"].transform(Z_test)
print(PC_test.shape)  # (3947, 11)
prediction = pipe["reg"].predict(PC_test)
```

Оба transform используют параметры, уже оценённые на X_train: средние, масштабы и направления главных компонент. X_test не получает свой scaler.fit или PCA.fit. Если обучить PCA отдельно на тесте, он выберет другой базис и регрессор получит координаты, несовместимые с обучающими. Кроме того, информация о тестовом распределении попадёт в настройку преобразования.

42. Нагрузки компонент и наиболее связанные исходные признаки

Таблица нагрузок отдельно не печатается в сокращённом notebook. Для защиты она дополнительно вычислена по финальному PCA:

```
pipe = pca_models["OLS"]
pca = pipe["pca"]
weights = pd.DataFrame(
    pca.components_.T,
    index=X_train.columns,
    columns=[f"PC{i+1}" for i in range(pca.n_components_)],
)
display(weights.iloc[:, :3])

Z = pipe["scale"].transform(X_train)
T = pca.transform(Z)
correlations = np.corrcoef(np.column_stack([Z, T]), rowvar=False)
feature_pc_correlations = correlations[:X.shape[1], X.shape[1]:]
```

Важно различать **веса направлений** и **корреляционные нагрузки**. components_ содержит собственные векторы: score компоненты — взвешенная сумма центрированных стандартизированных признаков. Понятие «нагрузка» также используют для корреляции признака с компонентой. Поэтому ниже явно приведены обе величины, а корреляции посчитаны напрямую.

Шесть наибольших по модулю весов для каждой из первых трёх компонент:

Компонента	Признак	Вес направления	Корреляция с PC
PC1	T9	0.304438	0.930542
PC1	T3	0.303719	0.928346
PC1	T1	0.300588	0.918776
PC1	T5	0.298823	0.913379
PC1	T4	0.297703	0.909956
PC1	T7	0.295031	0.901790
PC2	RH_4	0.358401	0.951722
PC2	RH_3	0.349311	0.927584
PC2	RH_7	0.347649	0.923173
PC2	RH_8	0.345972	0.918719
PC2	RH_9	0.337668	0.896668
PC2	RH_1	0.332108	0.881904
PC3	Windspeed	0.497421	0.669296
PC3	RH_out	-0.464627	-0.625170
PC3	T6	0.318379	0.428389
PC3	T_out	0.309732	0.416754
PC3	RH_2	-0.270929	-0.364543
PC3	T8	-0.247628	-0.333191

PC1 в основном описывает общий температурный режим, **PC2** — общую влажность, **PC3** содержит заметную погодную составляющую, включая ветер и наружную влажность противоположного знака. Это описательные названия направлений, а не установленные физические причины.

Матрица components_ имеет размер 11 × 25; таблица weights — 25 × 11. Одновременная смена знака всего собственного вектора и соответствующих координат не меняет содержание PCA. Поэтому знак компоненты сам по себе не является предметным выводом.

43. Проверка устранения мультиколлинеарности после PCA

Основной код: lab1.py, строки 147–151.

```
components = pd.DataFrame(pca_full.transform(scaled)[:, :k95],
                          columns=[f"PC{i+1}" for i in range(k95)])
vif_after = vif(components)
display(vif_after.round(6).to_frame())
pd.concat({"Исходные": vif_before, "PCA": vif_after}, names=["Базис", "Признак"]).to_csv(OUT /
↪  "vif.csv")
```

Рассчитывается тот же VIF, но теперь по координатам сохранённых компонент. **У всех 11 VIF≈1**: значения отличаются от единицы только на уровне погрешности арифметики с плавающей точкой.

Можно дополнительно проверить корреляции:

```
pc_corr = components.corr().to_numpy()
off_diagonal = np.abs(pc_corr - np.eye(k95))
print(off_diagonal.max())
```

На обучающем массиве значения вне диагонали близки к нулю. Ортогональные центрированные компоненты имеют диагональную выборочную ковариацию, поэтому одна линейно не предсказывается другими. Это не означает равенство их дисперсий единице и не доказывает статистическую независимость.

Гарантия относится к выборке обучения PCA. Дополнительная проверка на X_test дала максимальную по модулю межкомпонентную корреляцию около **0.0466**, то есть не строго ноль. Само преобразование при этом выполнено корректно обученным train-PCA.

44. Повторное обучение трёх регрессий на компонентах

Основной код: lab1.py, строки 163–166.

```
pca_metrics, pca_models = assess_family(use_pca=True)
comparison = pd.concat([base_metrics, pca_metrics], ignore_index=True)
comparison.to_csv(OUT / "metrics.csv", index=False)
display(comparison.round(4))
```

Вызов assess_family(use_pca=True) создаёт новые OLS, Lasso и Ridge, добавляет PCA в каждый Pipeline, выполняет тот же внутренний подбор и внешнюю CV, затем fit на полном train и predict теста.

Модели не получают заранее рассчитанный по всему train массив components: при CV это нарушило бы изоляцию фолдов. Они получают исходный X_train, а компоненты формируют внутри каждого своего обучающего конвейера. Финальные alpha после PCA: Lasso **0.1**, Ridge **31.622777**. Обученные варианты сохраняются в pca_models.

45. Одинаковы ли тестовые наблюдения и правила оценки

Да. Разбиение выполняется один раз до вызовов assess_family. Функция использует одни и те же X_train, X_test, y_train, y_test; повторного train_test_split внутри неё нет. Все шесть моделей оцениваются на одних и тех же **3947 индексах строк**.

Используются одинаковые внешние и внутренние KFold с фиксированными random_state, одни сетки alpha для каждого типа регрессии, одни формулы RMSE/R^2/MAPE и одна шкала цели Wh. Различается только наличие PCA; под каждое пространство параметры регуляризации подбираются заново.

results/predictions.csv сохраняет row_id, y_true и прогнозы всех шести вариантов. При проверке файла подтверждено, что каждый набор прогнозов имеет тот же состав теста и что все 18 тестовых метрик воспроизводятся непосредственно из этих прогнозов.

46. Итоговая таблица и эффект PCA для каждой модели

Тестовые результаты действующей лабораторной:

Признаки	Модель	RMSE, Wh	R^2	MAPE, %
Исходные	OLS	91.1717	0.169362	62.9209
Исходные	Lasso	91.1672	0.169442	62.8847
Исходные	Ridge	91.1681	0.169427	62.8994
PCA	OLS	93.9758	0.117482	65.0733
PCA	Lasso	93.9735	0.117524	65.0587
PCA	Ridge	93.9745	0.117506	65.0661

Изменение после PCA относительно соответствующего исходного варианта:

Модель	Δ RMSE, Wh	Δ R^2	Δ MAPE, п.п.
OLS	+2.804093	-0.051880	+2.152371
Lasso	+2.806234	-0.051918	+2.174082
Ridge	+2.806344	-0.051921	+2.166691

Для всех трёх регрессий PCA ухудшил все три тестовые метрики. RMSE вырос примерно на 2.8 Wh, R^2 уменьшился примерно на 0.052, MAPE увеличился примерно на 2.15–2.17 процентного пункта. На внешней CV картина та же: RMSE исходных моделей около 94.53 Wh, после PCA — около 97.62 Wh.

При этом PCA выполнил свою задачу: сократил размерность на 56% и устранил линейную мультиколлинеарность. Он выбирал направления по дисперсии X, не используя y. Часть отброшенных направлений с небольшой дисперсией оказалась полезна для предсказания расхода. Нельзя подгонять вывод под ожидание, что PCA обязан улучшить прогноз. Корректный итог: более компактные и ортогональные признаки получены, но за счёт умеренного ухудшения качества.

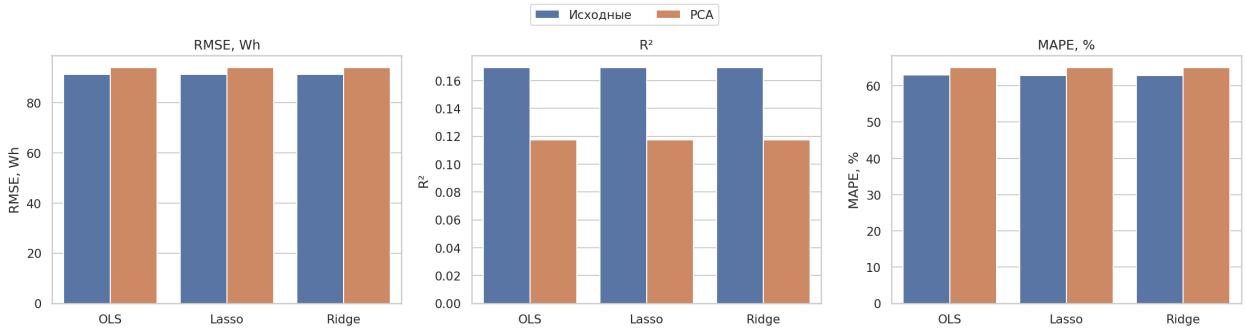


Рис. 4: Сравнение тестовых метрик до и после PCA

II. Теория

T1. Линейная регрессия

Линейная регрессия — метод обучения с учителем для предсказания количественной величины. Модель предполагает линейную по параметрам условную зависимость:

$$y_i = \beta_0 + \beta_1 x_{i1} + ... + \beta_p x_{ip} + \varepsilon_i, \qquad \hat{y}_i = \hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j x_{ij}.$$

Здесь x — известные предикторы, y — ответ, β — неизвестные параметры, ε — отклонение от модельной зависимости. В нашей работе p=25 до PCA; цель — энергия Appliances в Wh. Обычный OLS оценивает параметры минимизацией суммы квадратов остатков.

«Линейная» означает линейность по коэффициентам: если включить x^2 или взаимодействие x_1x_2 как отдельный заранее сформированный столбец, регрессия всё ещё будет линейной по параметрам. В текущей сокращённой версии таких дополнительных столбцов нет.

Вычислить OLS можно без предположения о нормальности всех признаков. Для обычных статистических гарантий важны корректность условного среднего $E(\varepsilon|X)=0$, отсутствие точной зависимости столбцов и условия на ошибки. При гомоскедастичных некоррелированных ошибках OLS имеет минимальную дисперсию среди линейных несмещённых оценок при условиях Гаусса–Маркова. Нормальность ошибок нужна для стандартных точных маловыборочных тестов и интервалов, а не для самого вычисления коэффициентов.

Линейность — приближение. Она может плохо описывать режимы включения приборов и редкие пики. Регрессионные коэффициенты без дополнительных условий исследования не являются причинными эффектами.

T2. Ridge-регрессия и L2-регуляризация

Ridge добавляет к квадратичной ошибке штраф за большие коэффициенты. В нормировке sklearn.Ridge:

$$L(\beta) = \sum_{i=1}^{n} (y_i - \beta_0 - x_i^T \beta)^2 + \alpha \sum_{j=1}^{p} \beta_j^2, \qquad \alpha \geq 0.$$

Свободный член не штрафуется. Для центрированных X и y решение имеет вид:

$$\hat{\beta} = (X^T X + \alpha I)^{-1} X^T y.$$

Добавление αI стабилизирует обращение плохо обусловленной матрицы, поэтому Ridge полезен при мультиколлинеарности. Он допускает некоторое смещение коэффициентов ради снижения их дисперсии — это компромисс смещения и разброса. При малом alpha решение близко к OLS; при очень большом коэффициенты стремятся к нулю, а прогноз — к среднему ответу при свободном члене.

L2 обычно плавно уменьшает веса, не превращая Ridge в метод отбора признаков. При коррелированных входах отдельный коэффициент может увеличиться или сменить знак: регуляризация ограничивает совокупную оценку, а не обязана уменьшать каждую координату по отдельности. Масштабирование важно: одинаковый физический эффект можно записать в метрах или миллиметрах с разными численными коэффициентами.

В нашей работе используется StandardScaler и solver="svd". Выбранные alpha: 3.1623 до PCA и 31.6228 после. Качество почти совпало с OLS: сам факт большого VIF не гарантирует, что регуляризация существенно улучшит прогноз на похожих данных.

T3. Lasso-регрессия и L1-регуляризация

Lasso использует абсолютные величины коэффициентов. В нормировке sklearn.Lasso:

$$L(\beta) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - x_i^T \beta)^2 + \alpha \sum_{j=1}^{p} |\beta_j|.$$

У L1-штрафа есть излом в нуле. Поэтому оптимум может содержать **точно нулевые коэффициенты**, то есть давать разреженное решение и выполнять отбор признаков. В отличие от обычного решения OLS, в общем случае нельзя просто записать одну формулу через обратную матрицу: используются численные методы, здесь — coordinate descent.

При сильной корреляции Lasso может оставлять один из похожих признаков и обнулять другой; выбор иногда чувствителен к данным. Перед штрафом нужна стандартизация. Параметр alpha выбирают по CV, а не по желаемому числу нулей после просмотра теста. Числа alpha Lasso и Ridge не сравниваются напрямую из-за разных нормировок ошибки и разных штрафов.

В нашем результате alpha=0.01 до PCA и 0.1 после. **Нулевых весов нет в обоих случаях**: возможность разреживания не означает обязательного отбора в конкретном запуске. Низкие штрафы оставили прогноз близким к OLS. Коэффициент при PC отражает связь с направлением, а не с отдельным датчиком; L1 также зависит от выбранной системы координат.

T4. Корреляционная матрица, мультиколлинеарность и VIF

Корреляционная матрица содержит парные коэффициенты связи переменных. У корреляции Пирсона r лежит от -1 до 1: положительное значение означает совместное линейное увеличение, отрицательное — линейное изменение в противоположных направлениях. Матрица симметрична, для непостоянного столбца на диагонали стоит 1.

Мультиколлинеарность — точная или приближённая линейная зависимость объясняющих столбцов. Она бывает многомерной: x_3 может быть близок к x_1+x_2, хотя ни одна парная корреляция не достигнет 0.99. Поэтому тепловой карты недостаточно; нужен VIF или другие диагностики обусловленности.

Для j-го признака:

$$VIF_j = \frac{1}{1 - R_j^2}.$$

Вспомогательный R_j^2 получают регрессией x_j на остальные X, со свободным членом. Цель основной задачи сюда не входит. VIF=1 означает отсутствие такой линейной объяснимости; значения >5 и >10 используются как ориентиры выраженной зависимости. При точной зависимости $R_j^2=1$ и VIF бесконечен.

При стандартных условиях дисперсия OLS-коэффициента содержит множитель VIF. Например, VIF≈150 означает сильное увеличение неопределённости коэффициента по сравнению с ортогональным дизайном при сопоставимых остальных условиях; стандартная ошибка изменяется примерно в √150≈12.25 раза. Это не «ошибка прогноза увеличилась в 150 раз».

Мультиколлинеарность сама по себе не делает OLS смещённым при корректном условном среднем, но делает оценки нестабильными и интерпретацию трудной. При точной зависимости коэффициенты не определяются единственным образом. Прогнозы при этом могут оставаться достаточно устойчивыми. Решения: предметное сокращение признаков, регуляризация, PCA, сбор более информативных данных. В работе VIF T_out≈150, после PCA все VIF≈1.

T5. Метод наименьших квадратов

МНК выбирает коэффициенты, минимизирующие сумму квадратов наблюдаемых ошибок:

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2.$$

В матричной записи X может включать столбец единиц для свободного члена. Приравнивая градиент к нулю, получаем нормальные уравнения:

$$X^T X \hat{\beta} = X^T y.$$

Если X имеет полный столбцовый ранг:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

Эта формула объясняет метод, но в программе не следует без необходимости явно вычислять inverse: численно устойчивее разложения и least-squares решатели. LinearRegression для плотных данных использует такую устойчивую линейную алгебру; при зависимости столбцов можно получить решение через псевдообратную матрицу.

Квадрат усиливает влияние крупных ошибок, поэтому редкие большие недооценённые пики заметно ухудшают нашу RMSE. Минимизация суммы квадратов, MSE и RMSE на одной фиксированной выборке без дополнительных штрафов даёт одинаковый оптимум: деление на n и квадратный корень не меняют порядок решений.

Для OLS со свободным членом на обучении остатки суммируются в ноль с точностью вычислений и ортогональны столбцам дизайна. Это не означает нулевые ошибки отдельных точек, гомоскедастичность или отсутствие ошибок на тесте. На тесте средний остаток может быть ненулевым.

T6. Коэффициент детерминации R²

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

SSE — сумма квадратов ошибок модели, SST — сумма квадратов отклонений ответов от среднего. Среднее в этой формуле относится к той выборке, на которой оценивается метрика.

- **$R^2=1$**: все ответы предсказаны точно.
- **$R^2=0$**: сумма квадратов ошибок равна ошибке постоянного прогноза среднего оцениваемых ответов.
- **$R^2<0$**: модель хуже этого среднего по квадратичной ошибке.

R^2 безразмерен. В нашей исходной Lasso тестовый $R^2 \approx 0.16944$: модель уменьшила сумму квадратов ошибок относительно вариации вокруг среднего тестовой цели примерно на 16.94%. Это не означает 16.94% «правильно классифицированных» интервалов и не означает среднюю ошибку 83.06%.

На тесте R^2 может быть отрицательным даже для нормальной вычислительно линейной модели. Добавление предикторов в нештрафной OLS на том же train не уменьшает обучающий R^2, но может ухудшить тестовый. Постоянная цель требует специального обращения со знаменателем; в нашем наборе дисперсия положительна. В общем случае R^2 не следует отождествлять с квадратом любой корреляции.

T7. Метрики RMSE, MAE и MAPE

$$RMSE = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2},$$

$$MAE = \frac{1}{n}\sum_i |y_i - \hat{y}_i|,$$

$$MAPE = \frac{100\%}{n}\sum_i \left|\frac{y_i - \hat{y}_i}{y_i}\right|.$$

RMSE выражена в единицах цели, здесь Wh. Она чувствительна к большим ошибкам из-за возведения в квадрат и подходит, когда пики особенно важны. **MAE** — средняя абсолютная ошибка в тех же Wh; она меньше усиливает редкие крупные промахи. На одной выборке без весов RMSE≥MAE. Дополнительный расчёт для нашей исходной Lasso дал MAE≈**52.53 Wh** при RMSE≈**91.17 Wh**: крупные ошибки имеют заметный вклад. MAE не была основной метрикой задания и не использовалась для выбора alpha.

MAPE — относительная ошибка в процентах. Она удобна для сопоставления масштабов, но плохо ведёт себя около нулевой цели и не определена математически при y=0 в указанной форме. Даже без нулей малые y делают вклад отдельных точек очень большим. Для Appliances минимум равен 10 Wh, а основной расход часто 50–100 Wh; MAPE≈63% следует интерпретировать с учётом этого распределения.

Разные функции потерь решают разные задачи: квадратичная ошибка ориентирована на условное среднее, абсолютная — на условную медиану. MAPE дополнительно перевзвешивает наблюдения через 1/|y|. Поэтому модели не обязаны иметь одинаковый рейтинг по трём метрикам. У нас основная метрика задана заранее — RMSE, остальные дополняют картину.

T8. Train, validation, test и кросс-валидация

Обучающая выборка используется для оценки параметров модели и предобработки: средних, масштабов, PCA-направлений, коэффициентов регрессии. **Валидационная** оценивает варианты гиперпараметров и помогает выбрать модель. **Тестовая** предназначена для итоговой оценки после выбора. Если выбирать параметры по тесту, он перестаёт быть независимой проверкой.

При K-fold CV данные делят на k фолдов. На каждой итерации один используется для оценки, остальные для обучения. Затем показатели усредняются. Это позволяет использовать разные наблюдения для проверки, но не делает ответы разных итераций полностью независимыми.

В нашей работе 20% — окончательный test, 80% — train для вложенной CV. Внутренние 3 фолда выбирают alpha; внешние 5 оценивают всю процедуру выбора. После этого модель переобучается на полном train. Отдельный фиксированный validation-набор не создаётся. Scaler и PCA входят в Pipeline, обучаемый внутри каждого фолда.

Случайная CV предполагает достаточную сопоставимость наблюдений. Для временного прогноза используют последовательные разбиения: прошлое обучает, более поздний участок проверяет. Для нескольких домов может понадобиться групповая проверка по дому. Наши случайные фолды одного временного ряда не обосновывают перенос на новые дома или будущие сезоны.

T9. Недообучение и переобучение

Недообучение возникает, когда модель или набор признаков недостаточно описывают существенные закономерности. Часто ошибки велики и на train, и на validation/test. Причины — слишком простой функциональный вид, потерянные полезные признаки или чрезмерная регуляризация.

Переобучение — приспособление к частностям обучающей выборки, включая шум. Типичный признак: очень хорошее обучение и заметно худшая проверка. Причины — избыточная гибкость относительно данных, подгонка гиперпараметров под проверочные ответы, утечки или малый train. Разрыв также может быть вызван сдвигом распределений, поэтому диагноз нельзя ставить только по одному сравнению ошибок.

Регуляризация и уменьшение размерности часто уменьшают разброс, но могут увеличить смещение. Слишком сильный alpha или слишком малое число PCA-компонент способны привести к недообучению. Для диагностики полезны кривые обучения, повторные корректные разбиения и проверка более выразительных спецификаций.

В нашей работе train RMSE≈94.33 и test≈91.17 Wh у исходной Lasso: выраженного переобучения по этому разрыву нет. После PCA выросла и обучающая, и тестовая ошибка. Это согласуется с потерей полезных направлений и ограничением выразительности, но не отменяет ограничений случайной временной проверки.

T10. Факторный анализ

Факторный анализ описывает наблюдаемые переменные через небольшое число скрытых общих факторов и индивидуальные компоненты:

$$x = \mu + \Lambda f + \varepsilon.$$

Здесь f — латентные факторы, Λ — матрица факторных нагрузок, ε — специфическая часть переменных. Типичная ковариационная модель:

$$\mathrm{Cov}(x) = \Lambda \Phi \Lambda^T + \Psi,$$

где Φ — ковариация факторов, Ψ — обычно диагональная матрица специфических дисперсий. Модель пытается объяснить общие связи, разделяя общую и индивидуальную вариативность. Для интерпретации нагрузок могут применяться вращения, например varimax; они в нашей работе не выполнялись.

Факторный анализ и PCA — разные методы. PCA строит детерминированные ортогональные комбинации X, последовательно объясняющие максимальную общую дисперсию. Классический факторный анализ вводит отдельную модель скрытых факторов и специфической ошибки и ориентирован на общую ковариационную структуру.

В задании раздел о факторном анализе реализован через PCA: отдельный sklearn.decomposition.FactorAnalysis не обучался. Температурную PC1 и влажностную PC2 можно описательно назвать обобщёнными направлениями, но нельзя утверждать, что доказано существование двух причинных физических латентных факторов.

T11. Метод главных компонент — PCA

PCA строит новый ортогональный базис, в котором первая ось объясняет максимальную дисперсию данных, вторая — максимальную оставшуюся при ортогональности первой, и так далее.

Для стандартизированной и центрированной матрицы Z:

$$C = \frac{Z^T Z}{n-1}, \qquad Cv_j = \lambda_j v_j,$$
$$\lambda_1 \geq \lambda_2 \geq \dots, \qquad T_k = ZV_k.$$

V_k содержит первые k собственных векторов, T_k — координаты наблюдений в пространстве компонент. Доля объяснённой дисперсии j-го направления равна $\lambda_j / \sum_l \lambda_l$. Практически sklearn здесь использует полный SVD, эквивалентный такому спектральному построению без обязательного явного вычисления C.

Усечённое представление сохраняет наиболее дисперсионные направления и минимизирует квадратичную ошибку восстановления центрированного Z среди соответствующих представлений ранга k. Это критерий восстановления **X**, не прогнозирования **y**. Поэтому малодисперсионная компонента может быть полезна для регрессии.

В нашем случае k=11 из 25 и 95.71% дисперсии X. Стандартизация нужна до PCA из-за разных единиц измерения. PCA центрирует, но сам не уравнивает масштабы признаков. Компоненты на train некоррелированы, но при whiten=False их дисперсии различаются; некоррелированность не равна статистической независимости.

Если сохранить все компоненты, ортогональный поворот сам по себе не меняет пространство линейных прогнозов OLS. L2-штраф Ridge также инвариантен к полному ортогональному повороту при неизменной шкале; L1-штраф Lasso зависит от ориентации осей. В нашей работе пространство усечено, поэтому изменилась и информация, доступная всем трём регрессиям.

T12. График каменистой осыпи

На горизонтальной оси показывают номер главной компоненты, на вертикальной — её собственное значение либо объясняемую долю дисперсии. Значения отсортированы по убыванию, поэтому кривая обычно падает. «Локоть» — переход от быстрого падения к более пологому хвосту — можно использовать как эвристический ориентир числа компонент.

Это не единственный и не всегда однозначный критерий. В нашей работе число **не выбиралось субъективно по локтю**: заранее задан порог накопленной дисперсии 95%. Накопленная кривая равна сумме explained_variance_ratio_ первых k осей.

Первые две оси сохраняют 65.58%, три — 72.82%. Несмотря на быстрое начальное падение, для принятого порога потребовались **11 осей**: 10 дают 94.7357%, 11 — 95.7086%. Поэтому ответ «выбрали 11, потому что там виден локоть» неверен. Верный ответ: «по минимальному числу компонент для порога 95%; слева показан спектр, справа накопление». График ничего напрямую не гарантирует относительно RMSE и R^2 по y.

T13. Зависимая и независимые переменные

Зависимая переменная — величина, которую требуется объяснить или предсказать. Она обозначается y; в работе это Appliances. **Независимые переменные** в терминологии регрессии — входные признаки X: температуры, влажности, освещение, погодные показатели.

Слово «независимые» здесь не означает, что предикторы статистически независимы друг от друга: как раз обнаружены сильные корреляции и большие VIF. Оно также не означает доказанное причинное влияние на y. Это традиционное название объясняющих переменных.

Строка X соответствует одному интервалу наблюдения и должна сопоставляться с тем же ответом y. Попадание y в X создаёт утечку. Передача будущего признака, недоступного в момент настоящего прогноза, может создать прикладную утечку даже при корректном программном train/test. У нас предполагается, что текущие измерения lights и погоды уже известны; это задача оценки расхода по текущему состоянию, а не готовая система сутки-вперёд.

T14. Остатки, гомоскедастичность и гетероскедастичность

Наблюдаемый остаток:

$$e_i = y_i - \hat{y}_i.$$

Его нужно отличать от истинной ненаблюдаемой ошибки ε_i в вероятностной модели. Положительный остаток означает недооценку расхода, отрицательный — завышение прогноза.

Гомоскедастичность: $\mathrm{Var}(\varepsilon|X) = \sigma^2$, условная дисперсия постоянна.
Гетероскедастичность: условная дисперсия зависит от X или уровня среднего. На

графике остаток–прогноз ищут изменение вертикального разброса, веер, кластеры и нелинейные рисунки. Горизонтальная линия 0 помогает видеть систематические отклонения.

В нашей работе OLS до PCA имеет стандартное отклонение остатков **44.97 Wh** в нижней группе прогнозов и **143.80 Wh** в верхней. После PCA — **36.74** и **136.33 Wh**. Это вместе с графиками свидетельствует о непостоянстве разброса. Нижняя наклонная граница частично обусловлена положительной целью: e≥min(y)−прогноз, поэтому одна форма границы недостаточна для диагноза. Формальный тест гетероскедастичности в текущей сокращённой работе не выполнялся; вывод имеет диагностический характер.

При корректном условном среднем гетероскедастичность сама по себе не делает OLS смещённым, но обычные формулы стандартных ошибок и интервалов могут быть некорректны; эффективность оценки ухудшается. Робастные HC-оценки ковариации учитывают неоднородную дисперсию, HAC также ориентированы на временную зависимость. Они изменяют оценку неопределённости, а не исправляют точечный прогноз. Возможны преобразование цели и взвешенный МНК при обоснованной модели дисперсии.

Нормальность ошибок, гомоскедастичность, отсутствие автокорреляции и мультиколлинеарность — разные свойства. PCA устраняет зависимости предикторов на train, но не гарантирует нормальные, независимые или гомоскедастичные остатки.



Рис. 5: Остатки исходных моделей

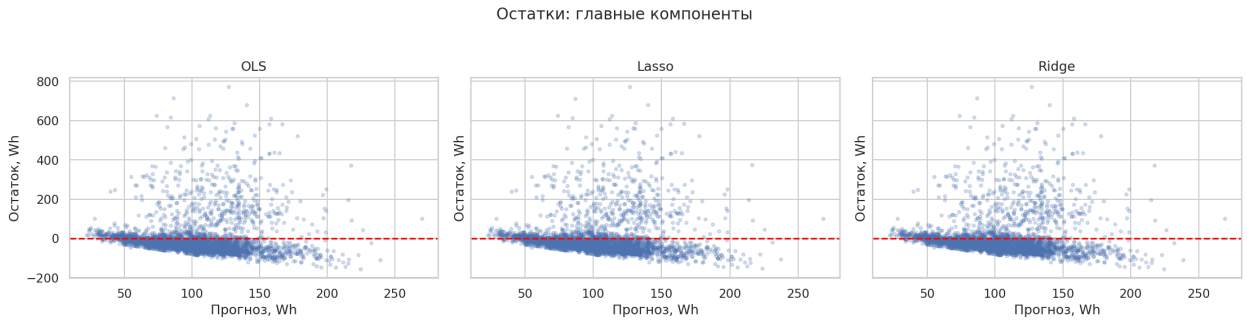


Рис. 6: Остатки моделей после PCA

T15. Коэффициенты, свободный член и множественная регрессия

Уравнение множественной линейной регрессии:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \ldots + \hat{\beta}_p x_p.$$

β_j — изменение условного линейного прогноза при увеличении x_j на единицу **при**

фиксированных остальных признаках. β_0 — прогноз при нулевых входах в выбранной системе координат. Его смысл зависит от того, достижима ли такая комбинация значений.

Наши регрессии получают стандартизированные z:

$$\hat{y} = \hat{\beta}_0 + \sum_j \hat{\beta}_j z_j, \qquad z_j = (x_j - \mu_j)/s_j.$$

Поэтому коэффициент относится к росту исходного признака на одно train-стандартное отклонение, а не на 1 °C или 1 процентный пункт. Для OLS $\beta_0 \approx 97.949709$ Wh — прогноз в центре стандартизированной обучающей таблицы, совпадающий со средним y_train.

Обратный переход к исходным единицам:

$$\beta_j^{\text{raw}} = \beta_j / s_j, \qquad \beta_0^{\text{raw}} = \beta_0 - \sum_j \beta_j \mu_j / s_j.$$

Например, стандартизированный коэффициент T_out у OLS равен −55.9471, а в исходной шкале — примерно **−10.5019 Wh/°C** при прочих равных. Первоначальный свободный член составляет 23.1937 Wh; нулевые влажности, давление и все температуры не описывают реалистичную типичную ситуацию, поэтому этому пересечению не придаётся самостоятельный физический смысл.

Высокая мультиколлинеарность мешает независимо менять коррелированные датчики и делает отдельные веса нестабильными. Отрицательный коэффициент температуры не доказывает, что нагрев конкретной комнаты причинно снижает потребление. Знаки могут зависеть от остальных включённых переменных.

После PCA модель имеет вид $\hat{y} = b_0 + \sum_k \gamma_k t_k$. Коэффициент γ_k относится к единице score компоненты. Влияния исходных стандартизированных координат можно восстановить через $\beta_Z = V_k \gamma$, где V_k — транспонированная components_. Для исходных физических единиц затем снова требуется деление на масштабы и корректировка свободного члена с учётом центрирования.

III. Короткая формулировка результата для начала защиты

«Я исследовал набор Appliances Energy Prediction — 19 735 десятиминутных измерений одного дома. Пропусков нет. В качестве цели взял Appliances в ватт-часах; исключил дату и два одинаковых искусственных случайных контроля, оставив 25 числовых предикторов.

Разделил данные 80/20. Масштабирование и PCA включил в Pipeline, а параметры Lasso и Ridge подбирал внутренней трёхфолдовой CV. Для внешней оценки использовал пять фолдов; окончательные метрики рассчитал на одних и тех же 3947 тестовых наблюдениях.

Обнаружил сильную мультиколлинеарность: у 17 предикторов VIF>10. PCA после стандартизации сохранил 11 компонент и 95.71% дисперсии X, снизив VIF компонент до 1. При этом качество ухудшилось: исходные модели дали RMSE около 91.17 Wh, после PCA — около 93.97 Wh. Это объясняется тем, что PCA оптимизирует сохранение дисперсии X, а не качество прогноза y.

Визуальная и групповая диагностика остатков показывает недооценку пиков и неодинаковый разброс. Между тремя регрессиями различия малы. Случайное

разбиение временных измерений ограничивает перенос вывода на прогноз будущих периодов».

Формулировки, которых следует избегать

- «Lasso обязательно удалила часть признаков» — у выбранной модели нулевых весов нет.
- «PCA сохранил 95% информации о цели» — речь о дисперсии признаков X.
- «VIF=1 доказывает полную независимость компонент» — это проверка линейных зависимостей на train.
- «Тест использовался для подбора alpha» — параметр выбирался на train во внутренней CV.
- «Я выполнил IQR-фильтрацию, временной тест или тест Бройша–Пагана в этой версии» — этих этапов в сокращённой лабораторной нет.
- «Высокий VIF делает прогноз обязательно плохим» — он прежде всего затрудняет устойчивую оценку отдельных коэффициентов.
- «MAPE=62.88% — это точность 37.12%» — это средняя абсолютная относительная ошибка, не доля правильных ответов.
- «R^2=0.1694 — модель угадала 16.94% строк» — коэффициент сравнивает суммы квадратов отклонений.